

Individuelle Praktische Arbeit

Inhaltsverzeichnis

1. Teil 1: Umfeld und Ablauf	3
1.1. Projektaufbauorganisation	3
1.1.1. Projektorganisation	3
1.1.2. Termine	3
1.1.3. Involvierte Personen	3
1.2. Zeitplan	4
1.3. Arbeitsprotokoll	5
1.3.1. Montag, 01.12.2025	5
1.3.2. Dienstag, 02.12.2025 (bis 12:00)	7
1.3.3. Mittwoch, 03.12.2025	8
1.3.4. Donnerstag, 04.12.2025	9
1.3.5. Freitag, 05.12.2025	10
1.3.6. Montag, 08.12.2025	11
1.3.7. Dienstag, 09.12.2025 (bis 12:00)	11
1.3.8. Mittwoch, 10.12.2025	12
1.3.9. Donnerstag, 11.12.2025	12
1.3.10. Freitag, 12.12.2025	13
1.3.11. Montag, 15.12.2025	14
2. Teil 2: Projekt	14
2.1. Informieren	14
2.1.1. Ausgangssituation	14
2.1.1.1. Screendesign	15
2.1.1.2. Detaillierte Aufgabenstellung	15
2.2. Planen	16
2.2.1. Verwendete Projektmanagementmethode	16
2.2.2. Versionierung und Datensicherheit	16
2.2.3. ERD	16
2.2.4. Routen	18
2.3. Entscheiden	21
2.4. Realisieren	21
2.5. Kontrollieren	21
2.6. Auswerten	21
2.6.1. Reflexion der Vorgehensweise	21
2.6.2. Bewertung des Produktes	21
2.6.3. Abweichungen zum Zeitplan	21
2.6.4. Persönliches Schlusswort und Bilanz	21
2.7. Quellenverzeichnis	21
3. Anhang	23
3.1. Git-Commit-History	23

1. Teil 1: Umfeld und Ablauf

1.1. Projektaufbauorganisation

1.1.1. Projektorganisation

Auszubildender	Lehrbetrieb
Sven Toye	twofold academy AG
Buchzelgstrasse 65	Thurgauerstrasse 54
8053 Zürich	8050 Zürich

1.1.2. Termine

Was?	Wann?
1. Expertenbesuch	Donnerstag, 04.12.2025
2. Expertenbesuch	
Präsentation, Demonstration, Fachgespräch	

1.1.3. Involvierte Personen

Person	Rolle
Artjan Illi	Hauptexperte
-	Nebenexperte
René Bach	Verantwortliche Fachkraft & Berufsbildner
Sven Toye	IPA-Prüfungskandidat

1.2. Zeitplan

				01.12 Mo				02.12 Di		03.12 Mi				04.12 Do				05.12 Fr				08.12 Mo				09.12 Di		10.12 Mi				11.12 Do				12.12 Fr				15.12 Mo			
IPERKA Phase	Aufgabe	Sollzeit	Istzeit	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60	62	64	66	68	70	72	74	76	78	80
(IPA)	Arbeitsjournal führen	34	20																																								
	Expertenbesuche	2	2																																								
Informieren	Informationen sammeln	2	2																																								
Planen	Planen + ERD erstellen	2	2																																								
Entscheiden	Lösungsvariante festlegen	2	2																																								
Realisieren	Entities	2	2																																								
	Requests	2	2																																								
	Services	2	2																																								
	Controllers	2	4																																								
	Relationen	2	6																																								
	Datenbank	2	2																																								
	Migrationen	2	2																																								
	Authentifizierung	4	4																																								
	Swagger	2	6																																								
Kontrollieren	Fehlerbehebung	6	8																																								
	Testen	4	12																																								
Auswerten	Reflexion & Fazit	2	0																																								
(Abschliessen)	Dokumentation abschliessen	2	2																																								
	Puffer	4	0																																								
Total		80	80																																								

1.3. Arbeitsprotokoll

1.3.1. Montag, 01.12.2025

Tagesziele gemäss Zeitplan	• Informieren und Aufgabenstellungen lesen • ERD erstellen • Zeitplan erstellen • Technologien auswählen • Git Repository erstellen • Dependencies herunterladen • Entities definieren
Ungeplante Arbeiten	• Entities erstellen ging schneller als gedacht. Datenbank Docker Container aufgesetzt am Ende des Tages aufgesetzt, um die Entities zu testen.
Erreichte Ziele	• Alle Ziele • Datenbank Docker Container aufgesetzt, aber wurde nicht fertig. • Ausgewählte Technologien: ▸ TypeScript (backend) ▸ NestJS (web framework) ▸ TypeORM (ORM) ▸ Node (Javascript runtime) ▸ PNPM (package manager) ▸ MySQL (Datenbank) ▸ PHPMYAdmin (Datenbank Dashboard) ▸ Docker (Containerization) ▸ Typst (Dokumente + Grafiken + Zeitplan + Diagramme) ▸ VSCode (IDE) ▸ Lazygit (Git Terminal UI)
Herausforderungen	• Ich hatte Probleme, die Lerndoku zu schreiben. Das Projekt ist nicht neu, aber die Lerndoku gleichzeitig schreiben doch schon und ich konnte mich nicht wirklich auf die Aufgaben konzentrieren.
Probleme	• Datenbank Verbindung scheiterte, weil die Umgebungsvariablen falsch waren/nicht angegeben waren. • Mehrere Errors und Exceptions, als ich den Dev-Server gestartet habe. Auslöser ist unklar.
Lösungen	• Das Datenbank Problem wurde am nächsten Arbeitstag gelöst. • Ich bin mir nicht sicher, wie ich das Problem mit dem Dev-Server gelöst habe, aber ich vermute, dass es etwas mit dem Datenbank Problem zu tun hatte.

Wissensbeschaffung	• NestJS und TypeORM Fähigkeiten verbessert. • Docker und Docker Compose verwendet, um die Datenbank aufzusetzen und zu konfigurieren.
Beanspruchte Hilfeleistung	• https://docs.nestjs.com/techniques/serialization • https://typeorm.io/docs/rerelations/many-to-one-one-to-many-relations • https://www.slingacademy.com/article/typescript-error-fix-the-types-have-no-overlap/ • https://typeorm.io/docs/entity/entities • https://docs.docker.com/reference/compose-file/services
Vergleich mit dem Soll-Zeitplan	Am Morgen war ich viel langsamer als geplant, weil ich mich nicht konzentrieren konnte, aber am Nachmittag war ich viel schneller.
Persönliche Tagesreflexion	Ich konnte mich nicht wirklich konzentrieren. Obwohl ich mehr erledigt als geplant habe, kam der Tag mir sehr unproduktiv vor.

1.3.2. Dienstag, 02.12.2025 (bis 12:00)

Tagesziele gemäss Zeitplan	Request DTOs definieren und erstellen.
Ungeplante Arbeiten	- Docker aufsetzen Docker vereinfacht die Konfiguration von der Datenbank, ohne lokal Software installieren oder konfigurieren zu müssen. - Datenbank Container - Datenbank Dashboard Container - Migrationen definieren - TypeORM CLI verwenden, um Migrationen zu generieren. - CSV Daten importieren.
Erfolgsereignisse	- Request DTOs erstellt. - Datenbank aufgesetzt mit Docker.
Herausforderungen	- Docker/Docker Compose aufsetzen und konfigurieren. - TypeORM CLI verwenden, um Migrationen erstellen.
Probleme	- Die gleichen Docker Probleme von gestern. - Die Build-Skripts hatte Kompatibilitäts-Probleme mit dem TypeORM CLI. - Die TypeORM CLI hat die Migrationen falsch generiert.
Lösungen	- Verschiedene Kombinationen von Umgebungsvariablen probiert und Ports geändert, bis es funktioniert hat. Ich bin mir aber nicht ganz sicher was das Problem war und es könnte sein, dass etwas anderes es auch beeinflusst hat. - Build-Skripts angepasst, sodass das TypeORM CLI funktioniert. - TypeORM CLI Migrationen-Kommando angepasst, sodass die Migrationen korrekt generiert werden.
Wissensbeschaffung	- Docker Compose Fähigkeiten verbessert. - Welche Umgebungsvariablen braucht, um den MySQL Container angepasst.
Beanspruchte Hilfeleistung	https://docs.docker.com/compose/how-tos/networking/ https://hub.docker.com/_/mysql/ https://dev.mysql.com/doc/refman/8.4/en/built-in-function-reference.html https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/#use-docker-compose https://typeorm.io/docs/migrations/why/#generating-migrations
Vergleich mit dem Soll-Zeitplan	Datenbank und Migrationen Aufgaben statt Request gelöst.
Persönliche Tagesreflexion	Ich konnte mich besser konzentrieren als gestern.

1.3.3. Mittwoch, 03.12.2025

Tagesziele gemäss Zeitplan	• Services erstellen. • Controllers erstellen. • Relationen definieren.
Ungeplante Arbeiten	• Requests erstellen. • Services erstellen. • Controllers erstellen. • Relationen definieren. • Serialisation implementieren.
Erfolgserlebnisse	• Relationen definiert und Relationen-Probleme gelöst. • Serialisation implementieren.
Herausforderungen	• Es gibt (und hat immer noch) mehrere Probleme mit den Relationen. Die meisten wurden gelöst, aber nicht alle.
Probleme	• Relationen von der Tabelle, die referenziert wird, bearbeiten, hat nicht funktioniert. Ein SquadPlayer besitzt einen Squad und ein Squad kann von mehrere SquadPlayers besitzt werden. Der Squad sollte den SquadPlayer, welcher ihn besitzt, bearbeiten können. • Serialisation hat nicht funktioniert, weil der ClassSerializerInterceptor nicht funktioniert hat. • Serialisation hat nicht funktioniert, weil es die Felder rekursiv in JSON umgewandelt hat. • Imports in der Datenbank-Konfiguration haben fehlgeschlagen.
Lösungen	• Ich habe Cascades verwendet, sodass man der Squad den SquadPlayer, welcher ihn besitzt, bearbeiten können.
Wissensbeschaffung	• Ich habe gelernt, wie man besser mit Relationen umgeht.
Beanspruchte Hilfeleistung	• https://typeorm.io/docs/relations/relations/#cascades • https://dev.to/mgohin/typeorm-remove-children-with-orphanedrowaction-4m7b • https://typeorm.io/docs/relations/many-to-one-one-to-many-relations • https://docs.nestjs.com/techniques/database#database
Vergleich mit dem Soll-Zeitplan	• Ich habe mehr erledigt als gedacht.
Persönliche Tagesreflexion	• Ich habe ganz viele Probleme gelöst, aber ich hatte nicht genug Zeit, das Arbeitsjournal zu führen.

1.3.4. Donnerstag, 04.12.2025

Tagesziele gemäss Zeitplan	• Datenbank
Erreichte Ziele	• Relationen verbessern. • Service verbessern, sodass sie mit den Relationen funktionieren. • Serialisation-Probleme lösen.
Erfolgserlebnisse	• Ich habe das letzte Problem mit den Relationen gelöst und es im Service umgesetzt. • Ich habe die Relationen-Probleme von gestern gelöst. • Ich habe die Serialisation-Probleme von gestern gelöst.
Herausforderungen	• Es gab noch ein letztes Problem mit den Relationen, dass ich gestern nicht lösen konnte.
Probleme	• Relationen im Service und im Controller umsetzen. • ClassSerializerInterceptor hat nicht funktioniert. • Die Fremdschlüssel waren nullable. • Die Fremdschlüssel hatten keine Datenbank On-Update-/On-Delete-Cascades.
Lösungen	• Fremdschlüssel im Entity anpassen und neue Migrations erstellen. • Interceptor im Controller hinzufügen. • Service anpassen, sodass die Relationen richtig funktionieren.
Wissensbeschaffung	• Wie man Relationen mit dem Service verwendet.
Beanspruchte Hilfeleistung	• https://typeorm.io/docs/entity/entities • https://typeorm.io/docs/relations/relations/#cascades • https://typeorm.io/docs/relations/many-to-one-one-to-many-relations • https://docs.nestjs.com/interceptors#binding-interceptors
Vergleich mit dem Soll-Zeitplan	• Die Datenbank war schon fertig und ich habe stattdessen die Probleme von gestern gelöst.
Persönliche Tagesreflexion	• Die Relationen und Serialisation funktionieren jetzt. • Ich habe Feedback vom Expertenbesuch im Arbeitsjournal gewendet.

1.3.5. Freitag, 05.12.2025

Tagesziele gemäss Zeitplan	• Migrationen definieren. • Authentifizierung
Erreichte Ziele	• Authentifizierung implementieren.
Herausforderungen	• JWT Guards im Controller einsetzen. • Konfiguration für Authentifizierung hat nicht funktioniert.
Probleme	• Die JWT Guards im Controller haben nicht funktioniert. • Git Commits gerebast statt gemergt. Die Commits waren im falschen Branch.
Lösungen	• UseGuards und die Authentifizierungs-Konfiguration angepasst. • Git Rebase Commits rückgängig gemacht und gemergt.
Wissensbeschaffung	• Wie man Authentifizierung mit JWT Guards implementiert.
Beanspruchte Hilfeleistung	• https://docs.nestjs.com/security/authentication
Vergleich mit dem Soll-Zeitplan	Ungefähr gleich, aber ich habe zu viel Zeit einplant.
Persönliche Tagesreflexion	• Authentifizierung fertig und einsatzbereit.

1.3.6. Montag, 08.12.2025

Tagesziele gemäss Zeitplan	• Authentifizierung • Swagger
Erreichte Ziele	• Swagger implementieren. • Controller verbessern, sodass es die Aufgabenstellung erspricht.
Erfolgserlebnisse	• Swagger implementieren. Swagger UI ist auf /api verfügbar. • Endpoints ▸ Request-Typ ▸ Response-Typ ▸ Parameter-Typ ▸ Beschreibung ▸ JWT Guards • DTOs ▸ Typ ▸ Beschreibung ▸ Beispiele
Herausforderungen	• Endpoints und DTOs mit Swagger dokumentieren. • Swagger Decorators
Probleme	• Swagger Decorators verwenden, um die Endpoints zu dokumentieren. • NestJS und Swagger Integration
Lösungen	• Decorators und Konfiguration angepasst und verbessert.
Wissensbeschaffung	• Swagger Fähigkeiten • Swagger mit NestJS Fähigkeiten (Decorators) • OpenAPI Spec
Beanspruchte Hilfeleistung	• https://docs.nestjs.com/interceptors • https://docs.nestjs.com/openapi/types-and-parameters • https://docs.nestjs.com/openapi/cli-plugin • https://docs.nestjs.com/openapi/operations
Vergleich mit dem Soll-Zeitplan	• Swagger Aufgaben gelöst statt Authentifizierung
Persönliche Tagesreflexion	• Swagger erfolgreich umgesetzt.

1.3.7. Dienstag, 09.12.2025 (bis 12:00)

Tagesziele gemäss Zeitplan	• Fehler-Behebung
Ungeplante Arbeiten	• Swagger-Dokumentation verbessern und erweitern • Relationen verbessert, sodass es die Aufgabenstellung entspricht. Ein Squad soll mehrere Trainer besitzen können.
Erreichte Ziele	• Swagger erweitert.

	• Relationen verbessert.
Erfolgserlebnisse	• Swagger • Relationen
Herausforderungen	• Relationen • Migrationen
Probleme	• Die Migrationen hatten Konflikte und ich konnte es nicht anpassen oder rückgängig machen.
Lösungen	• Migrationen rückgängig gemacht und neu erstellt.
Wissensbeschaffung	• Swagger • Relationen • Migrationen • TypeORM CLI
Beanspruchte Hilfeleistung	• https://docs.nestjs.com/openapi/types-and-parameters • https://docs.nestjs.com/openapi/introduction#setup-options • https://typeorm.io/docs/reactions/many-to-many-reactions#many-to-many-reactions-with-custom-properties
Vergleich mit dem Soll-Zeitplan	• Aufgabe war, Fehler zu finden und zu beheben, aber ich habe Swagger implementiert und die Relationen-Probleme gelöst.
Persönliche Tagesreflexion	• Viel Fortschritt in kurze Zeit

1.3.8. Mittwoch, 10.12.2025

Tagesziele gemäss Zeitplan	• Testing
Erreichte Ziele	• End-to-End Testing
Probleme	• Jest für NestJS einsetzen.
Lösungen	• Ich habe Deepseek gefragt, den Boilerplate zu generieren. • Imports angepasst.
Beanspruchte Hilfeleistung	• https://docs.nestjs.com/fundamentals/testing • https://stackoverflow.com/questions/60652617/how-to-mock-repository-service-and-controller-in-nestjs-typeorm-jest • https://stackoverflow.com/questions/55366037/inject-typeorm-repository-into-nestjs-service-for-mock-data-testing?rq=3
Vergleich mit dem Soll-Zeitplan	• Ungefähr gleich
Persönliche Tagesreflexion	• Nicht sehr viel Fortschritt

1.3.9. Donnerstag, 11.12.2025

Tagesziele gemäss Zeitplan	• Testing und Fehlerbehebung
----------------------------	--

Erreichte Ziele	• Testing gerefactored und automatisiert, sodass man die Tests nur ein Mal definieren kann und für jeden Endpoint wiederverwenden. • Fehler behoben. • Neue Tests für Spieler und Trainer erfolgreich implementiert.
Erfolgserlebnisse	• Testing verbessert, sodass ich die Tests wiederverwenden kann, ohne es für jeden Endpoint zu repetieren. • Fehler mit Tests gefunden und gelöst.
Probleme	• Tests refactoren.
Lösungen	• Bug lösen • Tests gerefactored.
Wissensbeschaffung	• Wie man E2E Tests implementiert.
Beanspruchte Hilfeleistung	• https://docs.nestjs.com/fundamentals/testing • https://stackoverflow.com/questions/60652617/how-to-mock-repository-service-and-controller-in-nestjs-typeorm-jest
Vergleich mit dem Soll-Zeitplan	• Ich habe die Tests verbessert, statt das Arbeitsjournal zu schreiben.
Persönliche Tagesreflexion	• Probleme mit den Tests überwunden und Fehler mit Tests gefunden.

1.3.10. Freitag, 12.12.2025

Tagesziele gemäss Zeitplan	• Puffer-Zeit • Arbeitsjournal führen
Ungeplante Arbeiten	• Tests verbessern. • Tests für SquadPlayer implementiert. • Fehler finden. • Fehler lösen.
Erfolgserlebnisse	• Tests für SquadPlayer verbessern. • Tests verbessern.
Herausforderungen	• Mock-Datenbank
Probleme	• Mock-Datenbank-Verbindung fehlgeschlagen.
Lösungen	• Imports angepasst. • App-Modul-Providers im Tests angepasst
Beanspruchte Hilfeleistung	• https://docs.nestjs.com/fundamentals/testing
Vergleich mit dem Soll-Zeitplan	• Ich hatte keine Zeit, das Arbeitsjournal zu führen.
Persönliche Tagesreflexion	• Sehr viele Fehler behoben, aber ich hatte keine Zeit, das Arbeitsjournal zu schreiben.

1.3.11. Montag, 15.12.2025

Tagesziele gemäss Zeitplan	• Arbeitsjournal führen • Reflexion und Fazit • Dokumentation abschliessen
Ungeplante Arbeiten	• Die Einträge im Arbeitsjournal nachholen, weil ich in den vergangenen Tagen nicht gemacht habe.
Erreichte Ziele	• Arbeitsjournal
Herausforderungen	• Arbeitsjournal abschliessen • Nicht genug Zeit
Vergleich mit dem Soll-Zeitplan	• Ich hatte nicht genug Zeit und die letzten drei Tagen vom Arbeitsjournal sind nicht sehr ausführlich.
Persönliche Tagesreflexion	• Die letzten Teil vom Arbeitsjournal fehlen teilweise oder sind nicht sehr ausführlich. Ich habe zu lange gewartet, es zu schreiben.

2. Teil 2: Projekt

2.1. Informieren

Eine REST API für die Verwaltung von Spieler*innen, Trainer*innen und Teamaufstellungen.

2.1.1. Ausgangssituation

Der Fussballverein des VfB Zürich-Leutschenbach verschickt zu jedem Spieltag seiner Jugendmannschaften eine Email an die Spieler/-innen und deren Eltern, in der der jeweilige Mannschaftskader bekannt gegeben wird.

Um diese Nachricht attraktiver zu gestalten und damit potentielle neue Spieler/-innen zu gewinnen, möchte der Vereinspräsident seinen Trainer/-innen eine WebApp zur Verfügung stellen, mit deren Hilfe sie den Kader des jeweiligen Spieles zusammenstellen können. Dieser Kader kann dann über eine definierte URL aufgerufen und per Email oder Whatsapp verschickt werden.

Spieler*innen und Trainer*innen können jeweils in mehreren Teamkadern gleichzeitig eingesetzt werden.

Teamkader müssen nicht aus Spieler*innen des gleichen Geschlechts bestehen, sondern dürfen auch gemischt sein.

Spieler*innen können verschiedene Positionen in unterschiedlichen haben.

Folgende Spielpositionen gibt es: Tor, Abwehr, Mittelfeld, Sturm, Ersatzspieler

Pro Teamkader können mehrere Trainer*innen gesetzt werden.

Aufgabe dieser PIPA ist es, eine dokumentierte REST Schnittstelle zur Verfügung zu stellen, die beim Bau der Frontend Applikation (NICHT TEIL DIESER API!) verwendet werden kann.

2.1.1.1. Screendesign

<https://miro.com/app/board/uXjVJupqp9M=?sharelinkid=685326180791>

2.1.1.2. Detaillierte Aufgabenstellung

Der Fokus dieser IPA liegt ausschliesslich auf der Erstellung einer REST API.

- Aufsetzen eines Gitlab Repositories, mit README, welches den Gebrauch der App beschreibt.
- Aufsetzen eines Nest.JS Projektes
- Einsatz von Code Qualitäts Tools (prettier, eslint)
- Programmiersprache ist TypeScript.
- Die Auswahl der Datenbank wird dem Kandidaten überlassen. Die Entscheidung muss in der Dokumentation erläutert werden.

Für einen potentiellen Frontend Entwickler, der die APP umsetzt, muss eine REST API erstellt werden. Die API wird mit Hilfe des Tools "Swagger" dokumentiert. Folgende Endpunkte müssen erstellt werden:

- /api/player, /api/player/:id
 - Felder: id, first_name, last_name, gender (male, female, diverse)
 - GET (protected): Auslesen einzelner und aller Spieler
 - POST (protected): Anlegen einzelner Spieler*innen
 - PUT (protected): Änderung einer bestehend Spieler*in
 - DELETE (protected): Löschen einer Spieler*in
- /api/trainer, /api/trainer/:id
 - Felder: id, first_name, last_name, gender (male, female, diverse)
 - GET (protected): Auslesen einzelner und aller Trainer*innen
 - POST (protected): Anlegen einzelner Trainer*innen
 - PUT (protected): Änderung einer bestehend Trainer*in
 - DELETE (protected): Löschen einer Trainer*in
- /api/squad, /api/squad/:id
 - Felder: id, description, date, player (list of players + positions), trainer (list of trainers)
 - GET all squad (protected): Liste aller existierenden Teamkader
 - GET single squad (public): Einzelner Teamkader
 - POST (protected): Neuen Squad anlegen
 - PUT (protected): Squad bearbeiten
 - DELETE (protected): Squad löschen

Endpunkte die mit "protected" gekennzeichnet sind, müssen im Request Header einen gültigen API-Key übermittelt bekommen. Ansonsten muss ein entsprechenden 400-er Response erfolgen.

Der Einfachheit halber, reicht es wenn ein statisch erstellter Token übergeben wird. Eine sonstige Authentifizierung bzw. Autorisierung ist nicht notwendig!

Endpunkte, die als "public" bezeichnet sind, benötigen keinen API-Key im Request Header.

Folgende Daten werden vom Verein zur Verfügung gestellt:

<https://git.twofold.dev/rbach/squad-app-preparation>

- CSV-Datei mit Spieler/-innen
- CSV-Datei mit Trainer/-innen

2.2. Planen

2.2.1. Verwendete Projektmanagementmethode

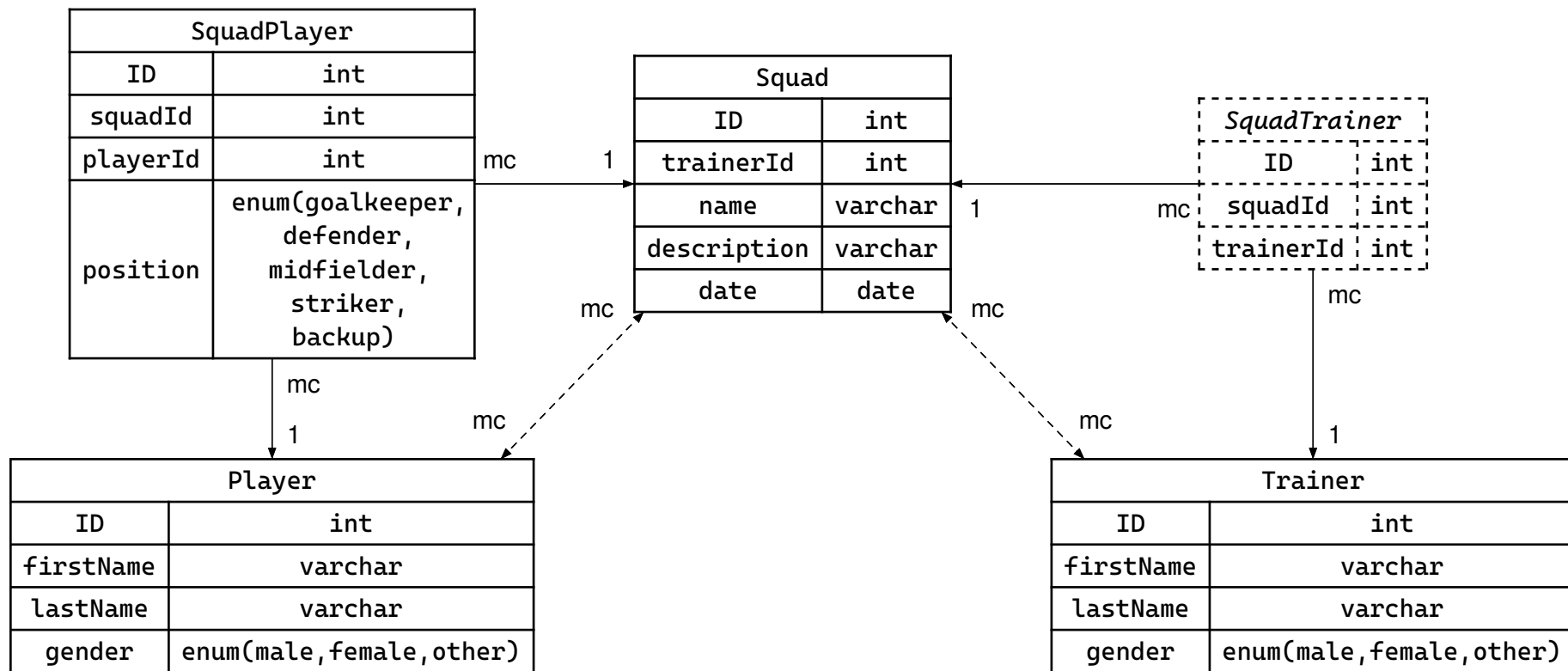
IPERKA

2.2.2. Versionierung und Datensicherheit

Repo: <https://git.twofold.dev/sventoye/squad-app-backend-pipa>

Wie ich Commits schreibe: <https://cbea.ms/git-commit/>

2.2.3. ERD



2.2.4. Routen

POST /api/auth/login	
Beschreibung	
Eingabe im <i>body</i>	object
Ausgabe 201	object
Ausgabe 401	void
POST /api/player	
Beschreibung	Erstelle einen Spieler
Eingabe im <i>body</i>	object
Ausgabe 201	object
Ausgabe 401	void
GET /api/player	
Beschreibung	Lese alle Spieler aus
Ausgabe 200	array
Ausgabe 401	void
GET /api/player/{id}	
Beschreibung	Lese einen Spieler aus
Eingabe <i>id</i> im <i>path</i>	number
Ausgabe 200	object
Ausgabe 401	void
Ausgabe 404	void
PATCH /api/player/{id}	
Beschreibung	Bearbeite einen Spieler
Eingabe <i>id</i> im <i>path</i>	number
Eingabe im <i>body</i>	object
Ausgabe 200	object
Ausgabe 401	void
Ausgabe 404	void
DELETE /api/player/{id}	
Beschreibung	Lösche einen Spieler
Eingabe <i>id</i> im <i>path</i>	number
Ausgabe 200	void
Ausgabe 401	void
Ausgabe 404	void
POST /api/trainer	

Beschreibung	Erstelle einen Trainer
Eingabe im <i>body</i>	object
Ausgabe 201	object
Ausgabe 401	<i>void</i>
GET /api/trainer	
Beschreibung	Lese alle Trainer aus
Ausgabe 200	array
Ausgabe 401	<i>void</i>
GET /api/trainer/{id}	
Beschreibung	Lese einen Trainer aus
Eingabe <i>id</i> im <i>path</i>	number
Ausgabe 200	object
Ausgabe 401	<i>void</i>
Ausgabe 404	<i>void</i>
PATCH /api/trainer/{id}	
Beschreibung	Bearbeite einen Trainer
Eingabe <i>id</i> im <i>path</i>	number
Eingabe im <i>body</i>	object
Ausgabe 200	object
Ausgabe 401	<i>void</i>
Ausgabe 404	<i>void</i>
DELETE /api/trainer/{id}	
Beschreibung	Lösche einen Trainer
Eingabe <i>id</i> im <i>path</i>	number
Ausgabe 200	<i>void</i>
Ausgabe 401	<i>void</i>
Ausgabe 404	<i>void</i>
POST /api/squad	
Beschreibung	Erstelle einen Squad
Eingabe im <i>body</i>	object
Ausgabe 201	object
Ausgabe 401	<i>void</i>
GET /api/squad	
Beschreibung	Lese alle Squad aus
Ausgabe 200	array

Ausgabe 401	<i>void</i>
GET /api/squad/{id}	
Beschreibung	Lese einen Squad aus
Eingabe id im <i>path</i>	<i>number</i>
Ausgabe 200	<i>object</i>
Ausgabe 404	<i>void</i>
PATCH /api/squad/{id}	
Beschreibung	Bearbeite einen Squad
Eingabe id im <i>path</i>	<i>number</i>
Eingabe im <i>body</i>	<i>object</i>
Ausgabe 200	<i>object</i>
Ausgabe 401	<i>void</i>
Ausgabe 404	<i>void</i>
DELETE /api/squad/{id}	
Beschreibung	Lösche einen Squad
Eingabe id im <i>path</i>	<i>number</i>
Ausgabe 200	<i>void</i>
Ausgabe 401	<i>void</i>
Ausgabe 404	<i>void</i>

2.3. Entscheiden

- TypeScript (backend)
- NestJS (web framework)
- TypeORM (ORM)
- Node (Javascript runtime)
- PNPM (package manager)
- MySQL (Datenbank)
- PHPMysqlAdmin (Datenbank Dashboard)
- Docker (Containerization)
- Typst (Dokumente + Grafiken + Zeitplan + Diagramme)
- VSCode (IDE)
- Lazygit (Git Terminal UI)

2.4. Realisieren

2.5. Kontrollieren

2.6. Auswerten

2.6.1. Reflexion der Vorgehensweise

Ich hatte Probleme, das Arbeitsjournal zu führen.

2.6.2. Bewertung des Produktes

Es entspricht die Aufgabenstellung.

2.6.3. Abweichungen zum Zeitplan

Ich hatte nicht genug Zeit, das Arbeitsjournal zu führen.

2.6.4. Persönliches Schlusswort und Bilanz

Das Projekt selber lief gut, aber ich musste das Arbeitsjournal in letzter Minute fertig machen.

2.7. Quellenverzeichnis

- <https://docs.nestjs.com/techniques/serialization>
- <https://typeorm.io/docs/rerelations/many-to-one-one-to-many-relations>
- <https://www.slingacademy.com/article/typescript-error-fix-the-types-have-no-overlap/>
- <https://typeorm.io/docs/entity/entities>
- <https://docs.docker.com/reference/compose-file/services>
- <https://docs.docker.com/compose/how-tos/networking/>
- https://hub.docker.com/_/mysql/
- <https://dev.mysql.com/doc/refman/8.4/en/built-in-function-reference.html>
- <https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/#use-docker-compose>
- <https://typeorm.io/docs/migrations/why/#generating-migrations>
- <https://typeorm.io/docs/rerelations/rerelations/#cascades>
- <https://dev.to/mgohin/typeorm-remove-children-with-orphanedrowaction-4m7b>

- <https://typeorm.io/docs/rerelations/many-to-one-one-to-many-relations>
- <https://docs.nestjs.com/techniques/database#database>
- <https://typeorm.io/docs/entity/entities>
- <https://typeorm.io/docs/rerelations/rerelations/#cascades>
- <https://typeorm.io/docs/rerelations/many-to-one-one-to-many-relations>
- <https://docs.nestjs.com/interceptors#binding-interceptors>
- <https://docs.nestjs.com/security/authentication>
- <https://docs.nestjs.com/interceptors>
- <https://docs.nestjs.com/openapi/types-and-parameters>
- <https://docs.nestjs.com/openapi/cli-plugin>
- <https://docs.nestjs.com/openapi/operations>
- <https://docs.nestjs.com/openapi/types-and-parameters>
- <https://docs.nestjs.com/openapi/introduction#setup-options>
- <https://typeorm.io/docs/rerelations/many-to-many-relations#many-to-many-relations-with-custom-properties>
- <https://docs.nestjs.com/fundamentals/testing>
- <https://stackoverflow.com/questions/60652617/how-to-mock-repository-service-and-controller-in-nestjs-typeorm-jest>
- <https://stackoverflow.com/questions/55366037/inject-typeorm-repository-into-nestjs-service-for-mock-data-testing?rq=3>
- <https://docs.nestjs.com/fundamentals/testing>
- <https://stackoverflow.com/questions/60652617/how-to-mock-repository-service-and-controller-in-nestjs-typeorm-jest>
- <https://docs.nestjs.com/fundamentals/testing>
- <https://miro.com/app/board/uXjVJupqp9M=?sharelinkid=685326180791>
- <https://git.twofold.dev/rbach/squad-app-preparation>
- <https://git.twofold.dev/sventoye/squad-app-backend-pipa>
- <https://cbea.ms/git-commit/>

3. Anhang

3.1. Git-Commit-History

```

[0] Commits - Reflog
8d4802bd 10:49 Sven Toye
ca3b7dbf 10:03 Sven Toye
37ce6a9b 12.12.25 Sven Toye
2993bc67 12.12.25 Sven Toye
b5f1d7db 12.12.25 Sven Toye
67d44846 12.12.25 Sven Toye
c71a928a 12.12.25 Sven Toye
9abcc602 12.12.25 Sven Toye
12e6cd8c 12.12.25 Sven Toye
03a9dd11 12.12.25 Sven Toye
146576b5 11.12.25 Sven Toye
8a5e3711 11.12.25 Sven Toye
57bab567 11.12.25 Sven Toye
c788e7f0 11.12.25 Sven Toye
a54ce483 11.12.25 Sven Toye
1146948b 10.12.25 Sven Toye
bb18431d 11.12.25 Sven Toye
5c288f9a 11.12.25 Sven Toye
e38faa33 10.12.25 Sven Toye
646ed526 10.12.25 Sven Toye
5b5e27bc 09.12.25 Sven Toye
8cbe2946 09.12.25 Sven Toye
4ede3b98 09.12.25 Sven Toye
bc4462ff 09.12.25 Sven Toye
ed464ce7 09.12.25 Sven Toye
47ab7bee 09.12.25 Sven Toye
1689240a 09.12.25 Sven Toye
8c8bb6d1 08.12.25 Sven Toye
9473cca2 08.12.25 Sven Toye
e7122076 08.12.25 Sven Toye
83ab0709 08.12.25 Sven Toye
27653ed2 08.12.25 Sven Toye
9ec35ae7 08.12.25 Sven Toye
09f4c670 08.12.25 Sven Toye
0722a7f4 08.12.25 Sven Toye
6bf9a034 08.12.25 Sven Toye
a947e352 04.12.25 Sven Toye
4344c867 02.12.25 Sven Toye
e348e9bf 08.12.25 Sven Toye
45b9dccc 04.12.25 Sven Toye
fe7665bf 04.12.25 Sven Toye
4bf7275fc 02.12.25 Sven Toye
d772efac 02.12.25 Sven Toye
76f3f9e6 01.12.25 Sven Toye
4d8802dc 02.12.25 Sven Toye
e9d89c42 01.12.25 Sven Toye
9dc94765 01.12.25 Sven Toye
43 of 45

o (HEAD -> develop, origin/develop) Update docs
o Update and refactor docs
o Update docs
o Merge branch 'fix/rename-trainer-field-to-trainers' into develop
o (fix/rename-trainer-field-to-trainers) Rename trainer field to trainers
o Replace api.json with openapi.yaml
o Update docs
o Format files
o Merge branch 'test' into develop
o (origin/test, test) Implement e2e tests for squad
o Update docs
o Update docs
o Make test suite generic over any entity and implement it for player and trainer
o Update docs
o Create e2e tests for player
o Add incomplete e2e tests
o Add baseUrl to openapi spec
o Add yaml openapi spec endpoint
o Update docs
o Update docs
o Merge branch 'fix/squad-trainers' into develop
o (origin/fix/squad-trainers, fix/squad-trainers) Make squad have multiple trainers
o Update docs
o Merge branch 'fix/responses' into develop
o (origin/fix/responses, fix/responses) Fix responses in controllers
o Merge branch 'feature/swagger' into develop
o (origin/feature/swagger, feature/swagger) Document endpoints, entities and DTOs in detail with swagger
o Document endpoints with swagger
o Merge branch 'feature/auth' into develop
o Add auth guards
o Add auth
o Update docs
o Update docs
o Update docs
o Add swagger endpoint
o Merge branch 'feature/relations' into develop
o (origin/feature/relations, feature/relations) Improve relations and relation management in services
o (origin/feature/migrations, feature/migrations) Create migrations
o Update docs
o Add docs in .pdf format
o Add docs
o (feature/docker) Fix database connection and network issues
o Add environment variables
o Add docker-compose.yml
o (fix/db-connection) Fix imports and config
o Create resources
o Set up project
o (origin/main, origin/HEAD, main) Initial commit

```